The Journal of Open Source Software

fastml: An R package for leakage-aware machine learning workflows with fold-local preprocessing

Selcuk Korkmaz [1¶], **Dincer Goksuluk** [2], **and Eda Karaismailoglu** [3]

1 Faculty of Medicine, Department of Biostatistics and Medical Informatics, Trakya University, Edirne 22030, Türkiye **2** Faculty of Medicine, Department of Biostatistics, Sakarya University, Sakarya 54290, Türkiye **3** Faculty of Medicine, Department of Medical Informatics, University of Health Sciences, Ankara 06010, Türkiye ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Henrik Finsberg

Reviewers:

- @prateekkohli2018-ampba
- @mcavs
- @ritika-giri

Submitted: 18 September 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Preprocessing leakage happens when a data-dependent transformation, such as scaling, imputation, class rebalancing or feature construction, is estimated on the whole dataset before resampling. Each fold's held-out observations have then already influenced the data the model trains on, and the performance estimate that follows is optimistic (Kaufman et al., 2012). This happens even for transformations that never touch the outcome labels (Moscovich & Rosset, 2022). Leakage more broadly is widespread. One survey of machine-learning-based science found it in 294 papers across seventeen fields (Kapoor & Narayanan, 2023), and in connectome-based neuroimaging, leakage through feature selection has been shown to inflate prediction performance substantially (Rosenblatt et al., 2024).

`fastml` is an R package that trains, tunes and compares machine-learning models through a single call. In `fastml`, *fold-local* preprocessing (preprocessing re-estimated independently within each fold) is the default execution path and not a convention the user has to remember to follow. Every data-dependent transformation is estimated inside each resampling split from that split's analysis data alone, then applied to the corresponding assessment data. The package covers classification, regression and survival tasks, and it offers resampling designs that respect grouping and time order alongside ordinary cross-validation. The evaluation design stays explicit and under the user's control.

Statement of need

Modular machine-learning frameworks break modelling into resampling, preprocessing, model specification and evaluation. Modularity is useful, but it also puts the burden of correct composition on the user, who has to apply the steps in the right order and keep training and assessment data strictly separated. Experienced practitioners still get this wrong in grouped, nested or time-ordered designs, because a leaky pipeline is a perfectly valid pipeline: it runs, it produces a number, and nothing in the output suggests that anything is wrong.

We wrote `fastml` for applied researchers who need a defensible performance estimate more than they need a maximally flexible pipeline. For that audience a silent error is expensive, and the freedom to compose a pipeline in any order is worth little. Every framework discussed below can express a safe workflow. The harder question is how easily each one can also express an unsafe workflow by accident.

`fastml` answers that by narrowing the interface. Its preprocessing arguments compile to untrained specifications that are estimated only inside the resampling loop. A recipe that has already been passed through `prep()` may carry parameters estimated outside that loop, so the

DRAFT

package rejects it before training begins. The restriction is deliberate; its costs and residual risks are set out under Limitations.

State of the field

Four R frameworks are the obvious points of comparison. `caret` (Kuhn, 2008) offers a legacy unified training interface. `tidymodels` (Kuhn & Wickham, 2020) emphasises explicit composition of recipes, model specifications, workflows and resampling. `mlr3` (Lang et al., 2019) provides an object-oriented task and learner design with a strong benchmarking culture, and `h2o` (Fryda et al., 2020) supplies an integrated AutoML system with its own training engine.

The comparison that matters most is with `mlr3pipelines` (Binder et al., 2021), because it couples preprocessing to evaluation in much the same way. Its `GraphLearner` composes preprocessing and a learner into a single object that is re-fitted on the training part of every fold, so fold-local preprocessing is already structural there and no separate guard is needed. `tidymodels` and `mlr3` can express leakage-safe workflows too. They can also express leakage-prone ones, just as easily and with no diagnostic.

`fastml` was developed not because existing frameworks cannot support safe workflows, but because using them safely depends on the user's configuration and expertise. Its main distinctions are the default workflow and the constraints it enforces. The package restricts supported pipeline structures so that preprocessing always stays inside the resampling loop, and it rejects unsupported designs rather than running an approximation that could compromise the validity of the estimate. Survival modelling calls for a more qualified comparison. Specialised frameworks such as `mlr3proba` (Sonabend et al., 2021) offer comparable or broader capabilities, and the contribution of `fastml` there lies in a consistent interface across heterogeneous learners rather than in greater methodological depth.

Software design

Under the guarded path, `fastml` works through each fold k in turn (Figure 1). It builds the split, fits a fresh preprocessing specification on the analysis data $D^{(k)}_{\text{analysis}}$ alone, applies that fold-trained specification to both $D^{(k)}_{\text{analysis}}$ and $D^{(k)}_{\text{assess}}$, and only then fits and evaluates the model. The assessment set is never used to estimate the transformation, only to be transformed by it. Tuning runs inside the same loop, so hyperparameter selection inherits the separation instead of undermining it.

DRAFT

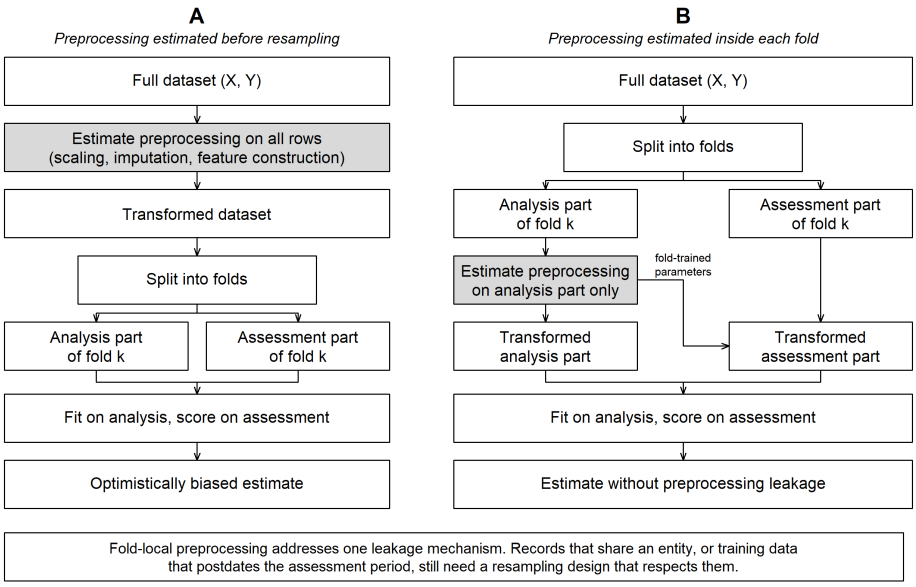


Figure 1: Where preprocessing is estimated. **(A)** Estimating a data-dependent transformation on the whole dataset before resampling lets each fold's assessment observations influence the data the model trains on, so the resulting score is optimistic. **(B)** The guarded path that `fastml` follows. The transformation is re-estimated from each fold's analysis part alone and then applied to both parts, so the assessment part is only ever transformed, never used to estimate the transformation itself.

The second design consideration concerned which requests the software should reject. Fold-local preprocessing addresses one source of leakage, but valid evaluation also requires a resampling design that accounts for dependencies in the data. For example, correlated observations from the same entity may be assigned to different folds (Roberts et al., 2017), or training observations may occur after the assessment period (Bergmeir & Benítez, 2012). `fastml` supports grouped, blocked and rolling resampling designs for these settings, and it returns an error when a requested design cannot be executed as specified. In particular, execution stops if `blocked_cv` is used without an ordering column, if `grouped_cv` is used without grouping columns, or if a design that requires ordered data receives observations that are not sorted in ascending order of the ordering variable.

The initial holdout split follows the same principles. When `data` and `test_size` are supplied together, the split is determined by the dependence structure the user has declared. If grouping columns are provided, entire groups are assigned to the holdout set. If an ordering column is provided, the final proportion of observations in temporal order is held out. Otherwise, the package uses a random row-wise split, stratified by the outcome for classification. This ensures that grouped cross-validation is never combined with a row-wise holdout.

Core functionality

A single `fastml()` call trains and compares models from 15 classification, 14 regression and 11 survival model families, mostly through established engines in the `tidymodels` ecosystem. The package supports seven resampling designs: ordinary and repeated cross-validation (`cv`, `repeatedcv`), the bootstrap (`boot`), `grouped_cv`, `blocked_cv`, `rolling_origin` and `nested_cv`. The `none` option provides holdout evaluation without resampling. Grouped, blocked and rolling designs accommodate dependent data, whereas nested cross-validation separates hyperparameter selection from performance estimation. This separation reduces the

DRAFT

selection bias that arises when the same folds are used for both purposes (Varma & Simon, 2006).

Survival modelling is supported through both `parsnip`-compatible engines and native implementations. The latter include the XGBoost accelerated failure time model (Barnwal et al., 2020) and a piecewise-exponential model implemented through `flexsurv` (Jackson, 2016). Additional features include exploratory diagnostics and model interpretation using permutation importance, accumulated local effects (ALE), LIME, individual conditional expectation (ICE) curves, surrogate trees and counterfactual explanations.

Example workflow

```r
library(fastml)

fit <- fastml(
  data       = my_data,
  label      = "outcome",
  algorithms = c("rand_forest", "logistic_reg"),
  resampling_method = "cv",
  folds      = 10
)

summary(fit)                # cross-validated comparison and the selected model
plot(fit, type = "roc")
predict(fit, newdata = new_rows)
```

Preprocessing options, including `impute_method`, `scaling_methods` and `balance_method`, are specified in the same call. Each option defines an untrained preprocessing step that is estimated using only the analysis portion of each fold. Dependence structure is declared in the same call: `group_cols` identifies grouping variables, while `block_col` defines the ordering variable for `blocked_cv` or `rolling_origin`. These settings determine both the initial holdout split and the resampling folds.

Limitations

The safeguards described above apply only inside a `fastml()` call, and three limitations remain.

First, the package cannot detect or undo preprocessing performed before the call. If the input data have already been scaled, imputed, rebalanced or filtered by feature selection using all observations, every fold inherits that leakage. Data-dependent preprocessing should be left to the package's own arguments or to an untrained recipe.

Second, the checks on user-supplied recipes are a safeguard, not a guarantee. `fastml` rejects trained recipes and recipes with steps that embed a data frame or reference the global environment, but it inspects a recipe's structure, not the origin of the values it holds. A custom step can therefore still carry a statistic computed from the full dataset, such as a mean or a list of selected features, that looks no different from a legitimate constant.

Third, the narrow interface excludes arbitrary pipeline structures, tuning of preprocessing parameters and stacking. Users who need these should turn to a modular framework and take responsibility for composing it correctly.

DRAFT

Research impact statement

`fastml` has been used in three peer-reviewed studies by research groups independent of its developers. Sun et al. (2026) used it to compare random forest, XGBoost, neural network and LightGBM models when building a global atlas of specific leaf area from 24,237 measurements of 5,687 vascular plant species. The analysis script they archived with their data calls `fastml()` directly (https://doi.org/10.6084/m9.figshare.29666732). Two clinical studies also used the package. Salgado-Garza et al. (2026) developed and internally validated models of increased healthcare utilisation among 7,535 patients who underwent colectomy for inflammatory bowel disease. Li et al. (2026) compared classification and regression models in a study of circulating microbial metabolites as predictors of tumour relapse and chemotherapy efficacy in nasopharyngeal carcinoma.

We also use the package in our own applied work. Yamasan & Korkmaz (2026) implemented every modelling step with `fastml` in a cross-platform blood-transcriptomics study of coronary artery disease, and its full analysis code is public. A separate methodological study by the developers (Korkmaz et al., 2026) evaluates the package through simulation and applied case studies. Since `fastml` first appeared on CRAN in November 2024, it has been downloaded about 109,000 times across 15 releases.

AI usage disclosure

We used Anthropic's Claude models through Claude Code during the development of `fastml` and the preparation of this manuscript. For software development, AI assistance supported defect identification, diagnosis and the drafting of code and tests. The authors wrote the manuscript and used AI solely to improve language clarity and concision. All package design decisions were made by the authors. Every AI-assisted software change was reviewed, tested and approved before inclusion, and all AI-assisted manuscript edits were checked by the authors. The authors take full responsibility for the software and the manuscript, including any remaining errors.

Conflicts of interest and funding

We declare no financial conflicts of interest. The work received no external funding.

Acknowledgements

We thank the users who reported issues against the package. Their feedback shaped its interface and its resampling behaviour.

References

Barnwal, A., Cho, H., & Hocking, T. D. (2020). *Survival regression with accelerated failure time model in XGBoost*. https://doi.org/10.48550/arXiv.2006.04920

Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences*, *191*, 192–213. https://doi.org/10.1016/j.ins.2011.12.028

Binder, M., Pfisterer, F., Lang, M., Schneider, L., Kotthoff, L., & Bischl, B. (2021). mlr3pipelines – flexible machine learning pipelines in R. *Journal of Machine Learning Research*, *22*(184), 1–7.

Fryda, T., LeDell, E., Gill, N., Aiello, S., Fu, A., Candel, A., Click, C., Kraljevic, T., Nykodym, T., Aboyoun, P., Kurka, M., & Malohlava, M. (2020). *h2o: R interface for the H2O*

DRAFT

scalable machine learning platform. https://CRAN.R-project.org/package=h2o

Jackson, C. (2016). flexsurv: A platform for parametric survival modeling in R. *Journal of Statistical Software*, *70*(8), 1–33. https://doi.org/10.18637/jss.v070.i08

Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, *4*(9), 100804. https://doi.org/10.1016/j.patter.2023.100804

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, *6*(4). https://doi.org/10.1145/2382577.2382579

Korkmaz, S., Goksuluk, D., & Karaismailoglu, E. (2026). *fastml: Guarded resampling workflows for safer automated machine learning in R*. https://doi.org/10.48550/arXiv.2604.05225

Kuhn, M. (2008). Building predictive models in R using the caret package. *Journal of Statistical Software*, *28*(5), 1–26. https://doi.org/10.18637/jss.v028.i05

Kuhn, M., & Wickham, H. (2020). *Tidymodels: A collection of packages for modeling and machine learning using tidyverse principles*. https://www.tidymodels.org

Lang, M., Binder, M., Richter, J., Schratz, P., Pfisterer, F., Coors, S., Au, Q., Casalicchio, G., Kotthoff, L., & Bischl, B. (2019). mlr3: A modern object-oriented machine learning framework in R. *Journal of Open Source Software*, *4*(44), 1903. https://doi.org/10.21105/joss.01903

Li, J.-Y., Yao, Y., Tan, X.-R., Si, N., Jiang, W., Lu, Y.-Q., Dai, J.-H., Yu, T.-T., Hu, H.-C., Duan, Y.-F., Feng, S.-Y., Huang, S.-W., Liang, Y.-L., Gong, S., Liu, N., Hu, Y.-M., & Li, Y.-Q. (2026). Circulating microbial metabolites predict tumor relapse and chemotherapy efficacy in nasopharyngeal carcinoma. *MedComm*, *7*(4), e70687. https://doi.org/10.1002/mco2.70687

Moscovich, A., & Rosset, S. (2022). On the cross-validation bias due to unsupervised preprocessing. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, *84*(4), 1474–1502. https://doi.org/10.1111/rssb.12537

Roberts, D. R., Bahn, V., Ciuti, S., Boyce, M. S., Elith, J., Guillera-Arroita, G., Hauenstein, S., Lahoz-Monfort, J. J., Schröder, B., Thuiller, W., Warton, D. I., Wintle, B. A., Hartig, F., & Dormann, C. F. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, *40*(8), 913–929. https://doi.org/10.1111/ecog.02881

Rosenblatt, M., Tejavibulya, L., Jiang, R., Noble, S., & Scheinost, D. (2024). Data leakage inflates prediction performance in connectome-based machine learning models. *Nature Communications*, *15*(1). https://doi.org/10.1038/s41467-024-46150-w

Salgado-Garza, G., Ballato, E., Araradian, C., Walsh, M., Fang, S. H., & Tsikitis, V. L. (2026). Machine learning prediction of increased healthcare utilization in colectomy for inflammatory bowel disease. *International Journal of Colorectal Disease*, *41*(1), 155. https://doi.org/10.1007/s00384-026-05184-w

Sonabend, R., Király, F. J., Bender, A., Bischl, B., & Lang, M. (2021). mlr3proba: An R package for machine learning in survival analysis. *Bioinformatics*, *37*(17), 2789–2791. https://doi.org/10.1093/bioinformatics/btab039

Sun, K., He, N., Xia, Y., Li, Y., & Xu, Z. (2026). A global atlas of specific leaf area under climate change. *Global Ecology and Biogeography*, *35*(1), e70193. https://doi.org/10.1111/geb.70193

Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, *7*(1), 91. https://doi.org/10.1186/1471-2105-7-91

Yamasan, B. E., & Korkmaz, S. (2026). Cross platform blood transcriptomics identifies a

DRAFT

two-gene classifier for coronary artery disease detection. *Namık Kemal Medical Journal*, *14*(2), 160–171. https://doi.org/10.4274/nkmj.galenos.2026.65807

DRAFT